

Closing the Triage Loop

MCP-mediated KLEE validation of
LLM-reported C/C++ vulnerabilities

Christopher Scherb, Alexander Trapp, Ruben Hutter,
Nico Bachmann, Luc Bryan Heitz

University of Applied Sciences and Arts Northwestern Switzerland

KES 2026 | **Dublin**

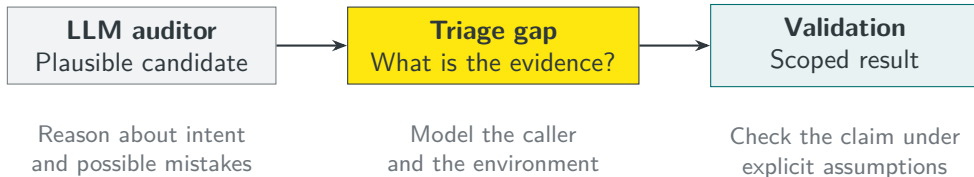
The AI says it found a bug

Would you report it?

- ▶ The explanation sounds plausible
- ▶ There is no observed failing execution yet
- ▶ The input assumptions may be wrong



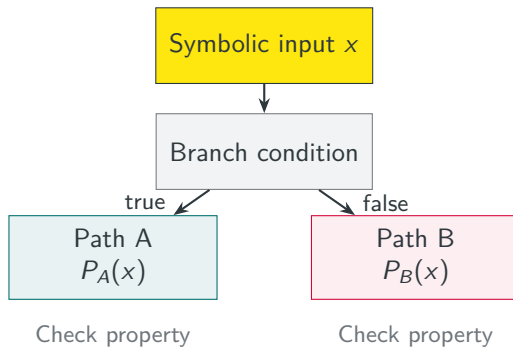
The triage gap



- ▶ LLMs propose candidates and infer context
- ▶ Symbolic execution checks behavior in an explicit model
- ▶ The harness and its preconditions connect the two

Symbolic execution in one minute

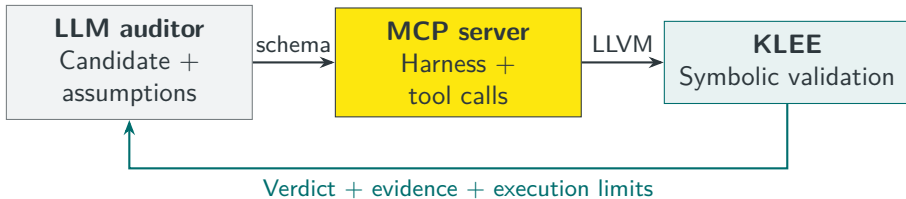
- ▶ Inputs represent sets of values
- ▶ Branches accumulate constraints
- ▶ A solver checks feasible paths
- ▶ An error path can yield a concrete witness



Every result depends on the harness and the execution model

The klee-mcp architecture

MCP = Model Context Protocol: a structured tool interface



- ▶ The LLM proposes the candidate and caller assumptions
- ▶ The server translates the specification into an analysis harness
- ▶ KLEE supplies the execution evidence

A candidate is an explicit specification

Candidate specification

Claim	Function and weakness class
Inputs	Symbolic parameters and buffer sizes
Contract	Assumptions about valid caller inputs
Run policy	Resource budget and retry choice



A machine-checkable
claim **before** analysis

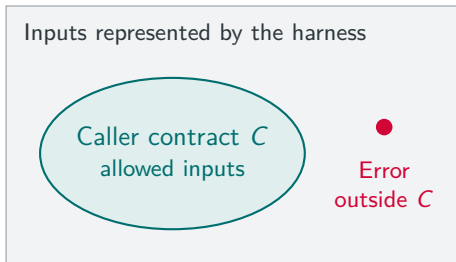
- ▶ Make the LLM's assumptions visible and reviewable
- ▶ Separate the suspected weakness from its input model

Reading the verdict

Confirmed	A matching error exists in the analyzed model
Infeasible	No matching error on the fully explored modeled paths
Unresolved	Timeout, build failure, or engine error prevents a conclusion

- ▶ Check assumptions, completed paths, and engine limitations
- ▶ A local witness alone does not establish deployment impact

Bounds encode a caller contract



$$C(x) \wedge \text{error}(x)$$

The query includes
the caller contract.

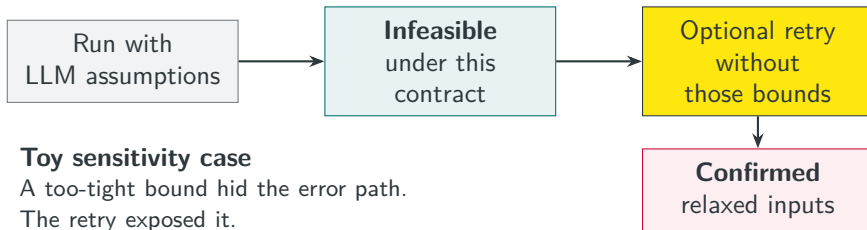
Changing C changes the claim

An error outside C does not refute
safety inside C .

- ▶ Bounds select which inputs the analysis considers possible
- ▶ A safe result inside that set says nothing about excluded inputs

Review bounds as part of the claim being verified

Retracting a questionable assumption



- ▶ Retain the initial verdict and record the relaxed retry
- ▶ A changed verdict exposes sensitivity to the assumption
- ▶ Review whether the relaxed inputs are possible for real callers

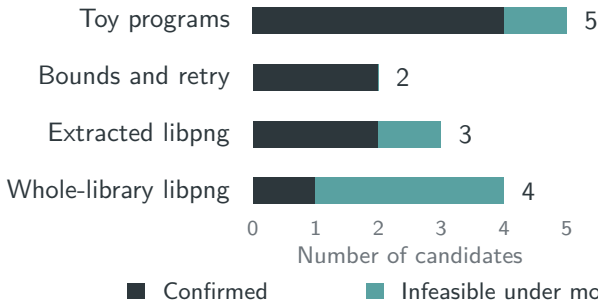
Proof-of-concept evaluation

14

hand-constructed
candidates

- ▶ Toy programs for selected weakness classes
 - ▶ Bounds and retry sensitivity cases
 - ▶ Extracted libpng functions
 - ▶ Whole-library libpng + zlib analysis
-
- ▶ KLEE 3.1; LLVM/Clang 13; STP + MiniSat
 - ▶ Whole-library target: 2.2 MB bitcode, 30 translation units

All 14 rows match their expected verdict

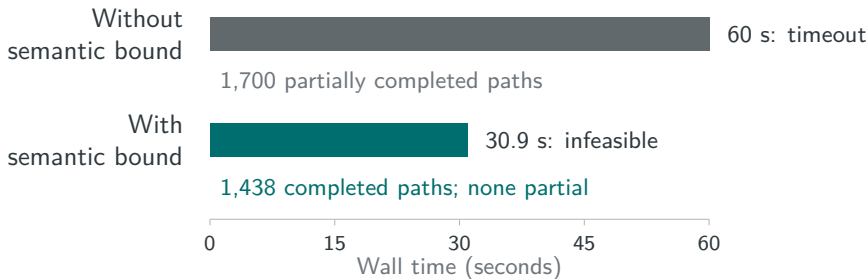


- ▶ 9 confirmed
- ▶ 5 infeasible
- ▶ 0.9–42.9 s per row

Evidence of internal consistency on these selected cases

- ▶ External precision, recall, and analyst time savings remain unmeasured

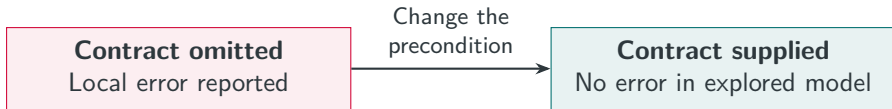
Bounds can make exploration finish



- ▶ Same libpng parser and harness; with and without a semantic bound
- ▶ The terminating result applies only under the added contract

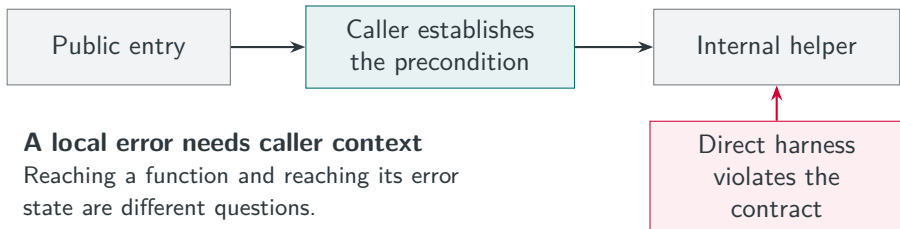
A historical CVE illustrates the contract

- ▶ CVE-2015-8540 case study: an internal libpng helper
- ▶ Function-level analysis reports an error across three versions
- ▶ Adding the caller contract changes the verdict to infeasible



Caller-side fixes can leave the helper's precondition essential

Reachability changes the interpretation



- ▶ `png_format_number`: an unchecked internal precondition
- ▶ Shipped callers satisfy the precondition at every reviewed call site
- ▶ Result: a defense-in-depth finding, with no internal error reachability

Where the evidence stops

- ▶ **Evaluation scope:** a small, selected benchmark; no external tool comparison
- ▶ **Model limits:** parser paths and `setjmp/longjmp` can limit exploration
- ▶ **Caller validity:** relaxing bounds does not establish a realistic execution
- ▶ **Impact:** heuristic labels and CVSS prefills need human assessment



What this work contributes

- ▶ **Explicit candidate schema**

A checkable claim with declared inputs and preconditions

- ▶ **Retractable bounds**

A second analysis exposes assumption-sensitive verdicts

- ▶ **Evidence for human review**

Execution results, caller context, and reporting artifacts

The LLM proposes the model; KLEE checks behavior within it

Before you report the next AI-found bug

- ▶ Ask for the execution evidence
- ▶ Ask which assumptions made the verdict possible
- ▶ Ask whether real callers can reach the error

A verdict needs a contract.

Questions?

github.com/FHNW-Security-Lab/klee-mcp

Backup: the benchmark rows

Candidate	Verdict	Wall time (s)
bof_01	confirmed	1.0
intoverflow_01	confirmed	2.4
null_01	confirmed	0.9
uaf_01	confirmed	0.9
safe_01	infeasible	1.1
bof_01_bounded	confirmed	0.9
bof_01_tootight	confirmed after retry	0.9
libpng_fp_number	infeasible	30.9
libpng_fp_number_bug	confirmed	5.9
libpng_fp_number_unb.	confirmed	9.4
libpng_sig_cmp	infeasible	5.1
libpng_fp_whole	infeasible	42.9
libpng_push_read	infeasible	4.1
libpng_format_number	confirmed; contract violation	4.2

Infeasible verdicts are conditional on the harness and engine model.

Backup: larger targets expose model limits

Target	Run result	Interpretation
libpng tRNS	240 s; timeout	Analysis unresolved
libjpeg-turbo	5.4 s; infeasible 1 completed path	Error handling limits exploration; weak evidence of safety
libxml2	23 s; infeasible 11 completed paths	A second whole-library run; still only the modeled inputs

- ▶ Whole-library linking is not whole-program coverage
- ▶ Read path statistics and unsupported behavior with the verdict

Backup: further reading

- ▶ Cadar, Dunbar, Engler (2008)
KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs, OSDI
- ▶ Ramos and Engler (2015)
Under-Constrained Symbolic Execution: Correctness Checking for Real Code, USENIX Security

Project: github.com/FHNW-Security-Lab/klee-mcp